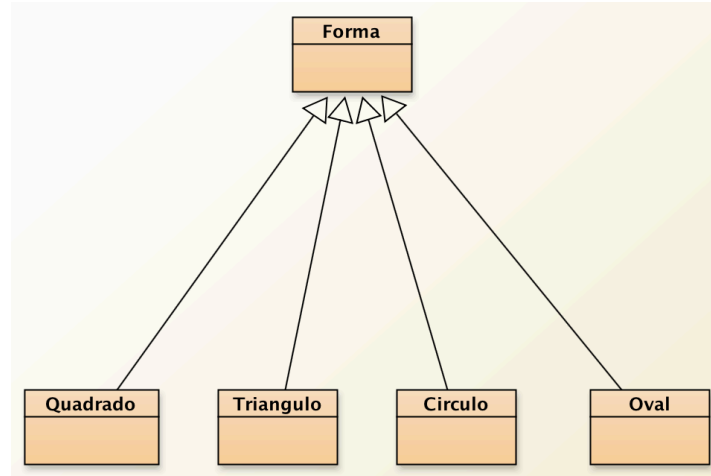


Classes Abstractas

- até ao momento todas as classes definiram completamente todo o seu estado e comportamento
- no entanto, por vezes, na concepção de soluções temos situações em que o código de uma classe pode não estar completamente definido
- esta é uma situação comum em POO e podemos tirar partido dela para criar soluções mais interessantes

- consideremos que precisamos de manipular forma geométricas (triângulos, quadrados e círculos)
- ... podemos acrescentar, com o evoluir da solução, mais formas geométricas
- torna-se necessário uniformizar a API que estas classes tem de respeitar
 - p. exemplo, todos devem possuir **area()** e **perimetro()**

- Seja então a seguinte hierarquia:



Qual a
motivação para criar
esta hierarquia??

- conceptualmente correcta e com capacidade de extensão através da inclusão de novas subclasses de Forma
- mas qual é o estado e comportamento de Forma?

- A classe Forma pode definir algumas v.i., como um ponto central (um Ponto), a espessura da linha, etc., mas se quisermos definir os métodos `area()` e `perimetro()` como é que podemos fazer?
- Solução I: não os definir deixando isso para as subclasses
 - as subclasses podem nunca definir estes métodos e aí perde-se a capacidade de dizer que todas as formas respondem a esses métodos

- Solução 2: definir os métodos `area()` e `perimetro()` com um resultado inútil, para que sejam herdados e redefinidos (!!?) - isto é uma solução? 🙄
- Solução 3: aceitar que nada pode ser escrito que possa ser aproveitado pelas subclasses e que a única declaração que interessa é a assinatura do método a implementar
 - a maioria das linguagens por objectos aceitam que as definições possam ser incompletas

- em POO designam-se por **classes abstractas** as classes nas quais, pelo menos, um método de instância não se encontra implementado, mas apenas declarado
 - são designados por **métodos abstractos** ou virtuais
 - uma classe 100% abstracta tem apenas assinaturas de métodos

- no caso da classe Forma não faz sentido definir os métodos `area()` e `perimetro()`, pelo que escrevemos apenas:

```
public abstract double area();  
public abstract double perimetro();
```

- como os métodos não estão definidos, a classe será também abstracta e não é possível criar instâncias de classes abstractas

- apesar de ser uma classe abstracta, o mecanismo de herança mantém-se e dessa forma uma classe abstracta é também um (novo) tipo de dados
 - compatível com as instâncias das suas subclasses
 - torna válido que se faça
Forma f = new Triangulo()

- uma classe abstracta ao não implementar determinados métodos, **obriga** a que as suas subclasses os implementem
- se não o fizerem, ficam como abstractas
- para que servem métodos abstractos?
- para garantir que as subclasses respondem àquelas mensagens de acordo com a implementação desejada

- Em resumo, as classes abstractas são um mecanismo muito importante em POO, dado que permitem:
- escrever especificações sintáticas para as quais são possíveis múltiplas implementações
- fazer com que futuras subclasses decidam como querem implementar esses métodos

- Na classe Circulo temos:

```
public double area() {  
    return Math.PI * Math.pow(this.raio,2);  
}
```

```
public double perimetro() {  
    return 2 * Math.PI * this.raio;  
}
```

- e em Rectangulo:

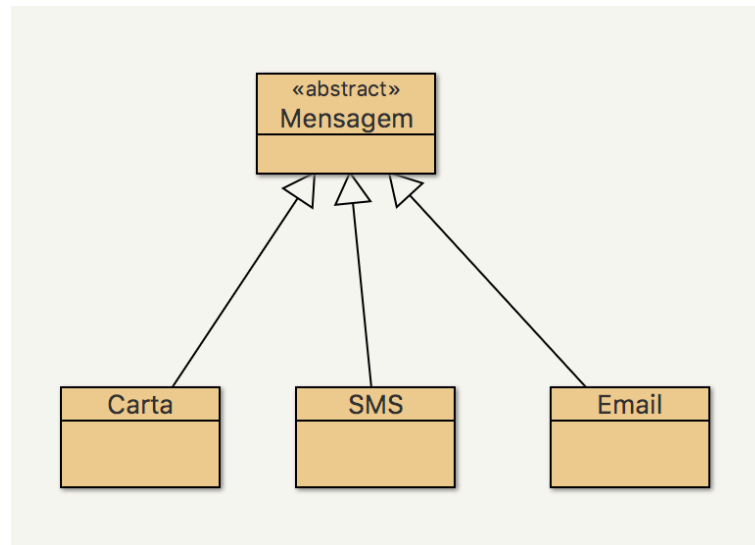
```
public double area() {  
    return this.ladoL * this.ladoA;  
}
```

```
public double perimetro() {  
    return 2 * this.ladoL + 2 * this.ladoA;  
}
```

- Podemos aproveitar a capacidade que os métodos abstractos proporcionam para impor comportamento às subclasses:

```
public abstract double area();  
public abstract double perimetro();  
public abstract String toString();  
public abstract FiguraGeometrica clone();
```

- Seja a seguinte hierarquia:



- cada uma das classes representa uma forma de mensagem. O que é comum a todas é a existência de uma variável “texto”

```
public abstract class Mensagem {  
    private String texto;  
  
    public Mensagem() {  
        this.texto = "";  
    }  
  
    public Mensagem(String texto) {  
        this.texto = texto;  
    }  
  
    public abstract String processa();  
  
    public String getTexto() {  
        return this.texto;  
    }  
  
    public void setTexto(String texto) {  
        this.texto = texto;  
    }  
}
```

```
public class Carta extends Mensagem {  
    private String enderecoOrigem;  
    private String enderecoDestino;  
  
    public Carta() {  
        super();  
        this.enderecoOrigem = "";  
        this.enderecoDestino = "";  
    }  
  
    public Carta(String remetente, String destinatario, String texto) {  
        super(texto);  
        this.enderecoOrigem = remetente;  
        this.enderecoDestino = destinatario;  
    }  
  
    public String processa() {  
        return "CARTA: Destinatário: " + this.enderecoOrigem  
            + "\nRemetente: " + "Mensagem: " + this.getTexto();  
    }  
}
```

```

public class SMS extends Mensagem {
    private String numeroOrigem;
    private String numeroDestino;

    public SMS() {
        super();
        this.numeroOrigem = "";
        this.numeroDestino = "";
    }

    public SMS(String nOrig, String nDest, String texto) {
        super(texto);
        this.numeroOrigem = nOrig;
        this.numeroDestino = nDest;
    }

    public String processa() {
        return ""+ this.numeroOrigem + ">> "
            + this.numeroDestino + "SMS: " + this.getTexto();
    }
}

```

```

public class Email extends Mensagem {
    private String emailOrigem;
    private String emailDestino;
    private String assunto;
    public Email() {
        super();
        this.emailOrigem = ""; this.emailDestino = ""; this.assunto = "";
    }
    public Email(String emailOrig, String emailDest, String assunto, String texto) {
        super(texto);
        this.emailOrigem = emailOrig;
        this.emailDestino = emailDest;
        this.assunto = assunto;
    }
    public String processa() {
        return "From :" + this.emailOrigem + "\nTo: " + this.emailDestino
            + "\nSubject: " + this.assunto + "\nTexto: " + this.getTexto();
    }
}

```

```
public class SistemaMensagens {  
    private List<Mensagem> mensagens;  
  
    public SistemaMensagens() {  
        this.mensagens = new ArrayList<>();  
    }  
  
    // ...  
  
    public int qtsEmails() {  
        return (int) this.mensagens.stream().filter(m -> m instanceof Email).count();  
    }  
  
    public int qtsDeTipo(String tipo) {  
        return (int) this.mensagens.stream().  
            filter(m -> m.getClass().getSimpleName().equals(tipo)).count();  
    }  
}
```

- o método `todasAsMensagens()` invoca o método polimórfico `processa()`, que é implementado de forma diferente em todas as classes

```
public String todasAsMensagens() {  
    StringBuilder sb = new StringBuilder();  
    sb.append("Todas as mensagens a enviar:\n");  
    for (Mensagem m: this.mensagens)  
        sb.append(m.processa()+"\n");  
  
    return sb.toString();  
}
```

- Classe de teste:

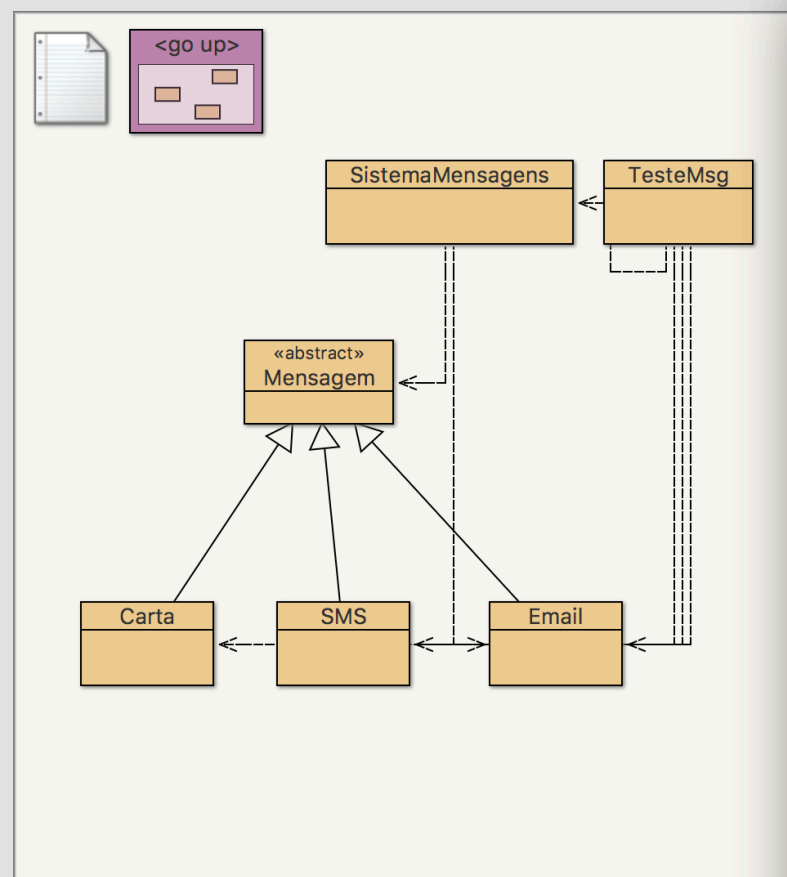
```
public static void main(String[] args) {
    SistemaMensagens sm = new SistemaMensagens();

    Carta c1 = new Carta("José Francisco", "Pedro Xavier", "Em anexo a proposta de compra.");
    Carta c2 = new Carta("Produtos Estrela", "Joana Silva", "Junto enviamos factura.");
    SMS s1 = new SMS("961234432", "929745228", "Estou à espera!");
    SMS s2 = new SMS("911254535", "939541928", "Hoje não há aula...");
    Email e1 = new Email("anr", "jfc", "Teste P00", "Junto envio o enunciado.");
    Email e2 = new Email("a77721", "a55212", "Apontamentos", "Onde estão as fotocópias?");
    Email e3 = new Email("anr", "a43298", "Re: Entrega Projecto", "Recebido.");

    sm.addMensagem(c1); sm.addMensagem(c2);
    sm.addMensagem(s1); sm.addMensagem(s2);
    sm.addMensagem(e1); sm.addMensagem(e2); sm.addMensagem(e3);

    System.out.println("Número de Emails: " + sm.qtsEmails());
    System.out.println("Número de SMS: " + sm.qtsDeTipo("SMS"));

    System.out.println(sm.todasAsMensagens());
}
```



Número de Emails: 3

Número de SMS: 2

Todas as mensagens a enviar:

CARTA: Destinatário: José Francisco

Remetente: MENSAGEM: Em anexo a proposta de compra.

CARTA: Destinatário: Produtos Estrela

Remetente: MENSAGEM: Junto enviamos factura.

961234432>> 929745228SMS: Estou à espera!

911254535>> 939541928SMS: Hoje não há aula...

From :anr

To: jfc

Subject: Teste P00

Texto: Junto envio o enunciado.

From :a77721

To: a55212

Subject: Apontamentos

Texto: Onde estão as fotocópias?

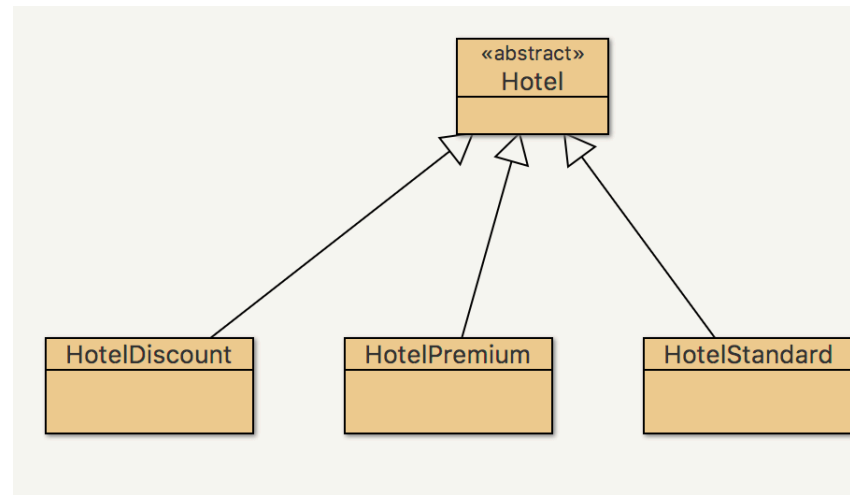
From :anr

To: a43298

Subject: Re: Entrega Projecto

Texto: Recebido.

- Na hierarquia:



- ...o método que determina o preço de um quarto é abstracto. A sua concretização é feita em cada uma das subclasses.

```
/**
 * Método que determina o valor de uma noite de um quarto.
 * Este método é abstracto na classe Hotel e implementado em
 * cada uma das subclasses.
 */
public abstract double precoNoite();
```

- HotelStandard:

```
/**
 * Calcula o preço de uma noite no hotel
 * @return valor aumentado da taxa de época alta (se for o caso)
 */

public double precoNoite() {
    return getPrecoBaseQuarto() + (epocaAlta?20:0);
}
```

- HotelDiscount:

```
/**
 * Calcula o preço de uma noite no hotel
 * @return valor do preço base afectado pela ocupação.
 */

public double precoNoite() {
    return getPrecoBaseQuarto() * 0.75 + getPrecoBaseQuarto() * 0.25 * ocupacao;
}
```

○ equals, novamente...

- de acordo com a estratégia anteriormente apresentada, o método equals de uma subclasse deve invocar o método equals da superclasse, para nesse contexto comparar os valores das v.i. lá declaradas.
- utilização de **super.equals()**

- seja o método equals da classe Aluno (já conhecido de todos)

```
/**
 * Implementação do método de igualdade entre dois Aluno
 *
 * @param umAluno    aluno que é comparado com o receptor
 * ** * @return      booleano true ou false
 * ** */
public boolean equals(Object umAluno) {
    if (this == umAluno)
        return true;

    if ((umAluno == null) || (this.getClass() != umAluno.getClass()))
        return false;
    Aluno a = (Aluno) umAluno;
    return (this.nome.equals(a.getNome()) && this.nota == a.getNota()
        && this.numero == a.getNumero());
}
```

- seja agora o método equals da classe AlunoTE, que é subclasse de Aluno:

```
/**
 * Implementação do método de igualdade entre dois Alunos do tipo T-E
 *
 * @param umAluno aluno que é comparado com o receptor
 * ** * @return booleano true ou false
 * ** */
public boolean equals(Object umAluno) {
    if (this == umAluno)
        return true;

    if ((umAluno == null) || (this.getClass() != umAluno.getClass()))
        return false;
    AlunoTE a = (AlunoTE) umAluno;
    return(super.equals(a) & this.nomeEmpresa.equals(a.getNomeEmpresa()));
}
```

- considerando o que se sabe sobre os tipos de dados, a invocação **this.getClass()** continua a dar os resultados pretendidos?

Herança vs Composição

- Herança e composição são duas formas de relacionamento entre classes
- são no entanto abordagens muito distintas e constitui um erro muito comum achar que podem ser utilizadas da mesma forma
- existe uma tendência para se confundir herança com composição

- quando uma classe é criada por composição/agregação de outras, isso implica que as instâncias das classes compostas/agregadas fazem parte da definição do contentor
- é uma relação do tipo “parte de” (part-of)
- qualquer instância da classe vai ser constituída por instâncias das classes compostas/agregadas
- Exemplo: Círculo tem um ponto central (Ponto)

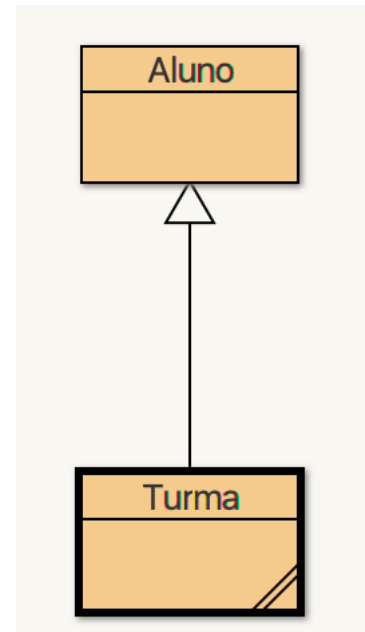
- do ponto de vista do ciclo de vida a relação é fácil de estabelecer:
- (quando é composição) quando a instância contentor desaparece, as instâncias agregadas também desaparecem
- o seu tempo de vida está iminentemente ligado ao tempo de vida da instância de que fazem parte!
- (quando é agregação) desaparece a relação entre os objectos

- esta é uma forma (...e está aqui a confusão!) de criar entidades mais complexas a partir de entidades mais simples:
- Turma é composta por instâncias de Aluno
- Automóvel é composto por Pneu, Motor, Chassis, ...
- Clube é composto por instâncias de Atleta, Funcionario, Dirigente, ...

- quando uma classe (apesar de poder ter instâncias de outras classes no seu estado interno) for uma especialização de outra, então a relação é de **herança**
- quando não ocorrer esta noção de especialização, então a relação deverá ser de **composição/agregação**

- Uma forma simples de testar se faz sentido a relação ser de herança consiste em “ler” o diagrama.

- Por exemplo:

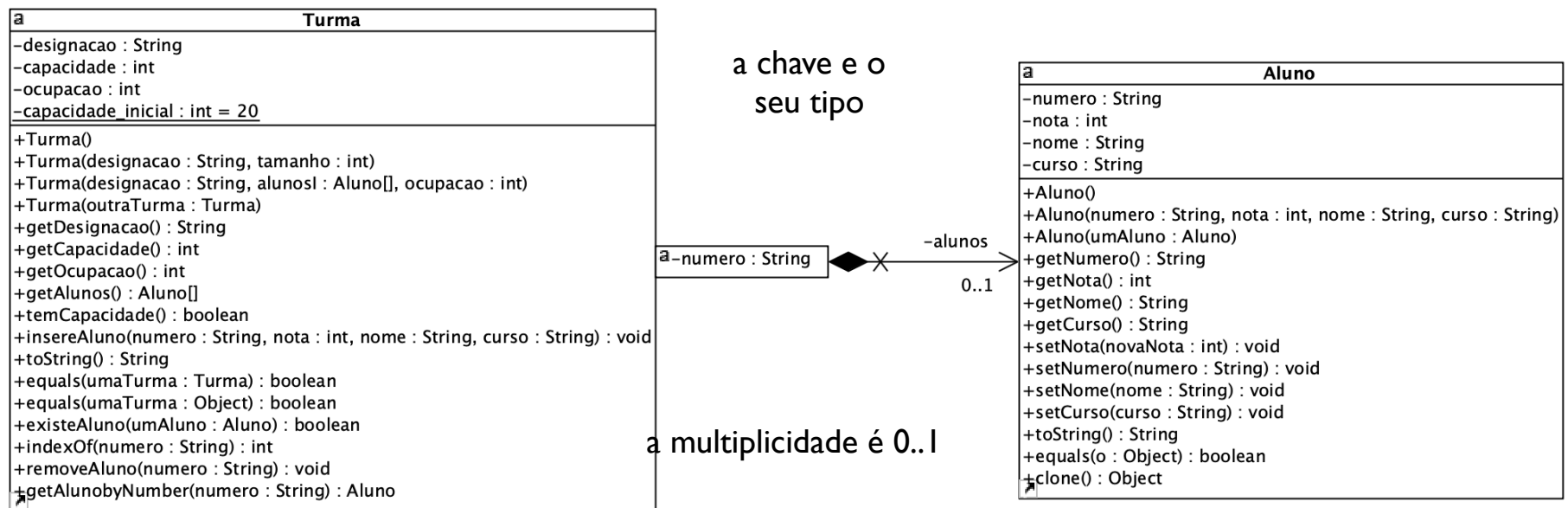


- nunca será possível esta estruturação, pois não é verdade que “uma turma é um aluno”!

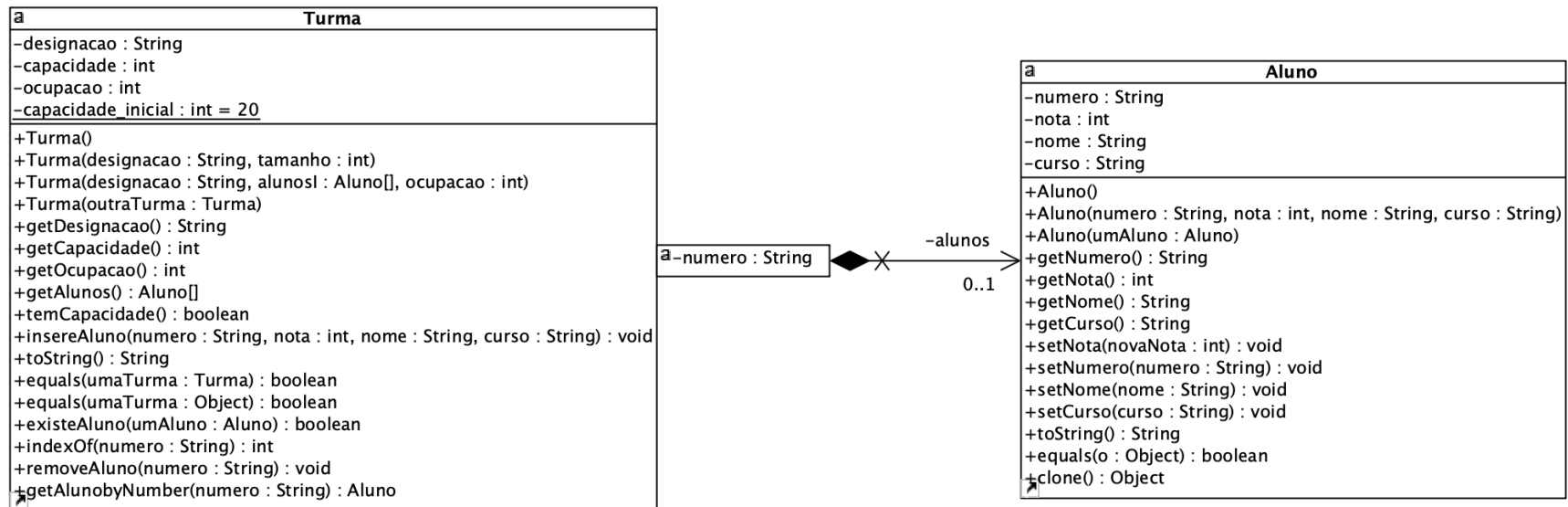
Notação do Diagrama de Classe UML

- Em relação ao diagrama de classe UML que temos vindo a utilizar é necessário acrescentar mais informação:
 - como descrever apropriadamente mapeamentos
 - como descrever o que é abstracto

- Na descrição do Map vamos indicar a chave e o seu tipo e a classe dos objectos que fazem parte dos valores.



- se a chave existir temos acesso a uma instância de Aluno, caso contrário a zero!



- dá origem a:

```

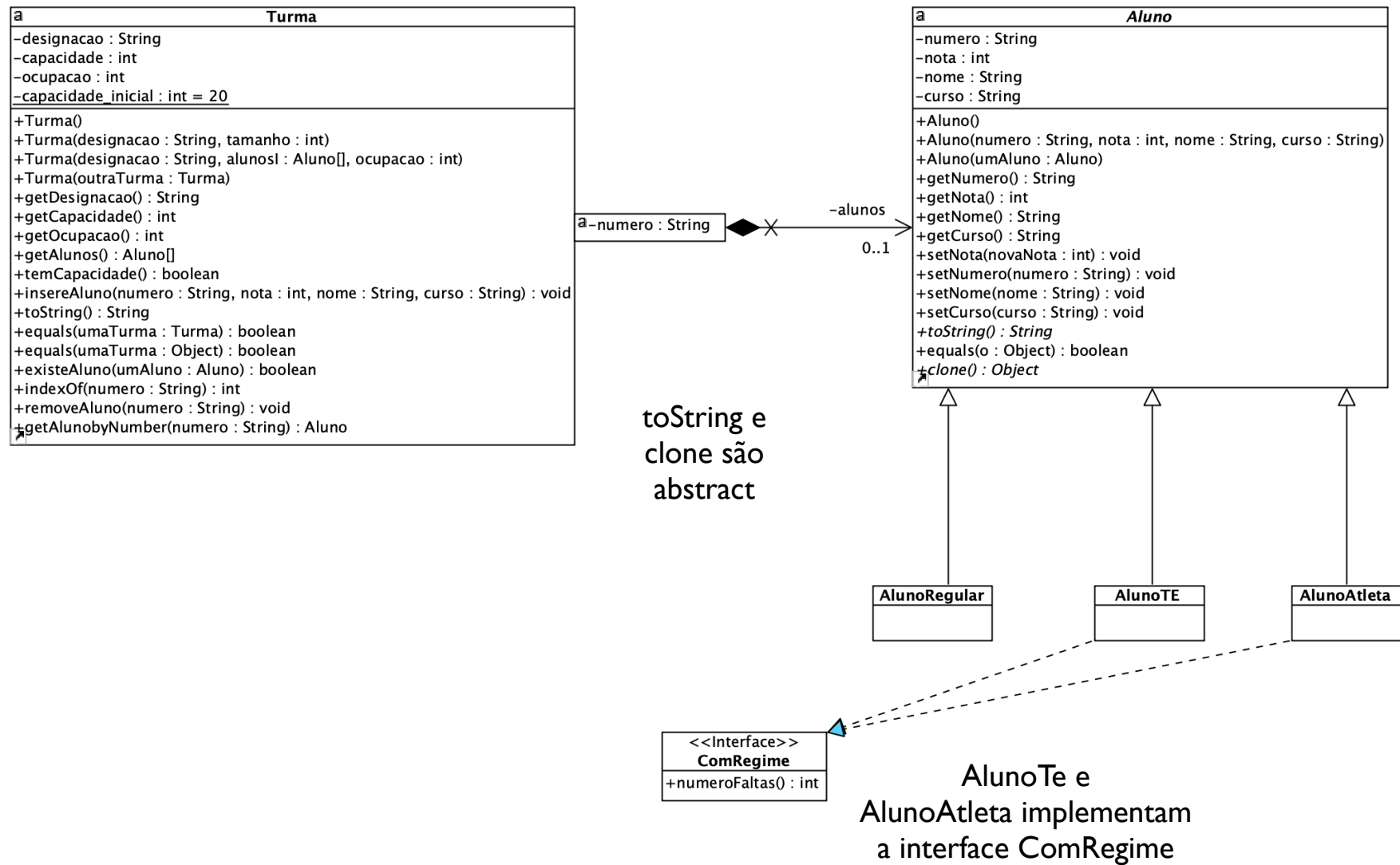
public class Turma {
    private String designacao;
    private int capacidade;
    private int ocupacao;
    private static final int capacidade_inicial = 20;

    private Map<String, Aluno> alunos;

    ...
  
```

- Em UML as definições que são abstractas são anotadas a *itálico* ou em alguns editores como utilizando a notação <<abstract>> (já não faz parte da norma...)

Aluno é
abstract



toString e
clone são
abstract

AlunoTe e
AlunoAtleta implementam
a interface ComRegime

Interfaces

- Como vimos atrás, as classes além de descreverem estrutura e comportamento, podem ser também vistas como tipos de dados
- um tipo de dados representa o comportamento que é possível invocar num objecto desse tipo
- O tipo de dados básico de uma instância é o tipo da classe que a criou (através da invocação do construtor)

- Na declaração:

```
ClasseA a = new ClasseA();  
ClasseB b = new ClasseB();
```
- os objectos a e b são do tipo de dados que lhes é dado pela classe que os cria.
- Os tipos de dados das subclasses podem ser designados como subtipos e já sabemos que:
 - se ClasseA é superclasse de ClasseB
 - um objecto do tipo ClasseA pode ser substituído por um objecto do subtipo ClasseB sem alteração nas classes que manipulam objectos do tipo ClasseA.

- Este mecanismo de substituição é essencial para as construções polimórficas que estudamos anteriormente.

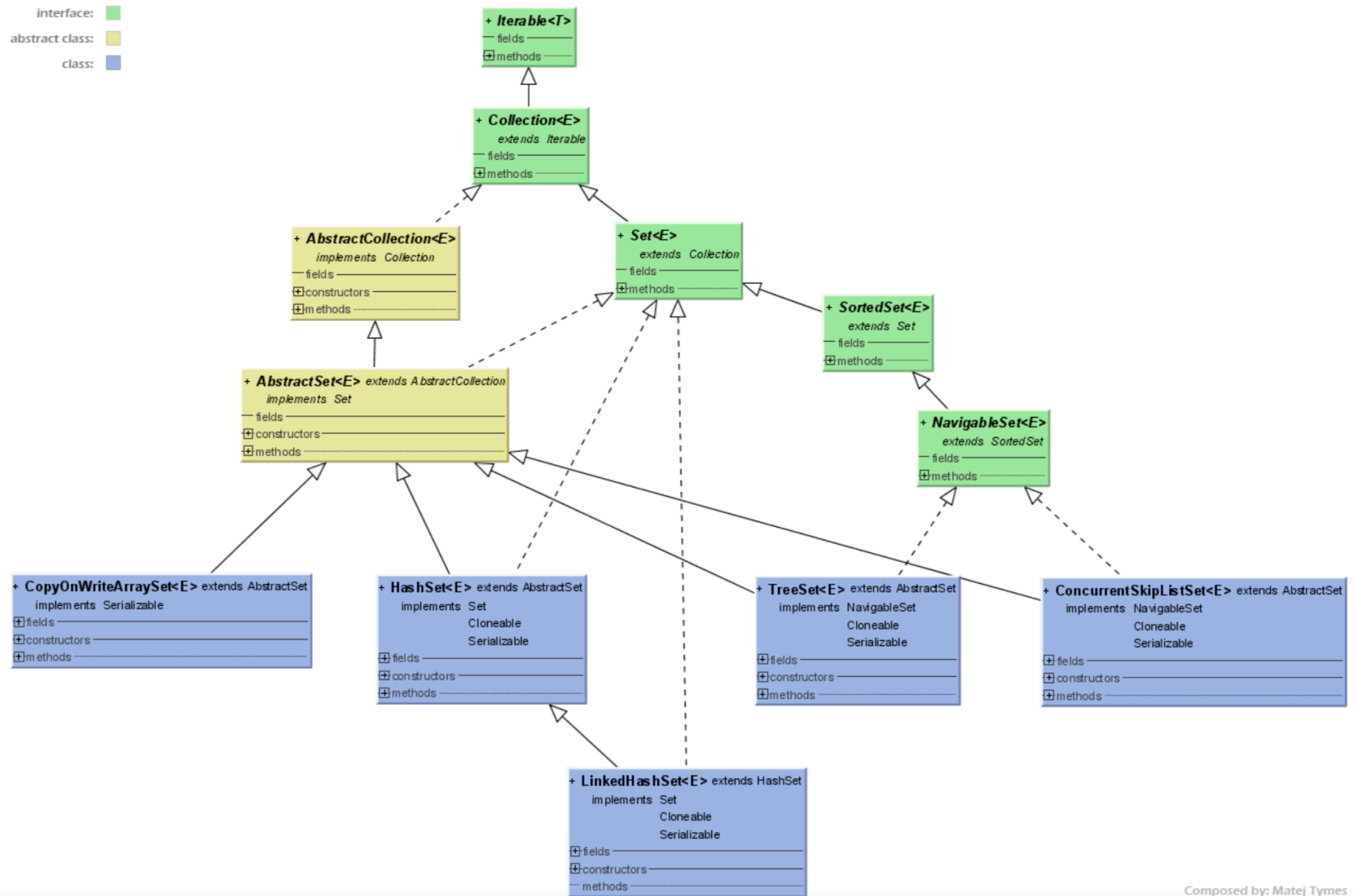
```
Mensagem c1 = new Carta("José Francisco", "Pedro Xavier", "Em anexo a proposta de compra.");  
Mensagem c2 = new Carta("Produtos Estrela", "Joana Silva", "Junto enviamos factura.");  
Mensagem s1 = new SMS("961234432", "929745228", "Estou à espera!");  
Mensagem s2 = new SMS("911254535", "939541928", "Hoje não há aula...");  
Mensagem e1 = new Email("anr", "jfc", "Teste P00", "Junto envio o enunciado.");  
Mensagem e2 = new Email("a77721", "a55212", "Apontamentos", "Onde estão as fotocópias?");  
Mensagem e3 = new Email("anr", "a43298", "Re: Entrega Projecto", "Recebido.");
```

- Claro que esta capacidade de substituição de tipos por subtipos só funciona no contexto de uma hierarquia

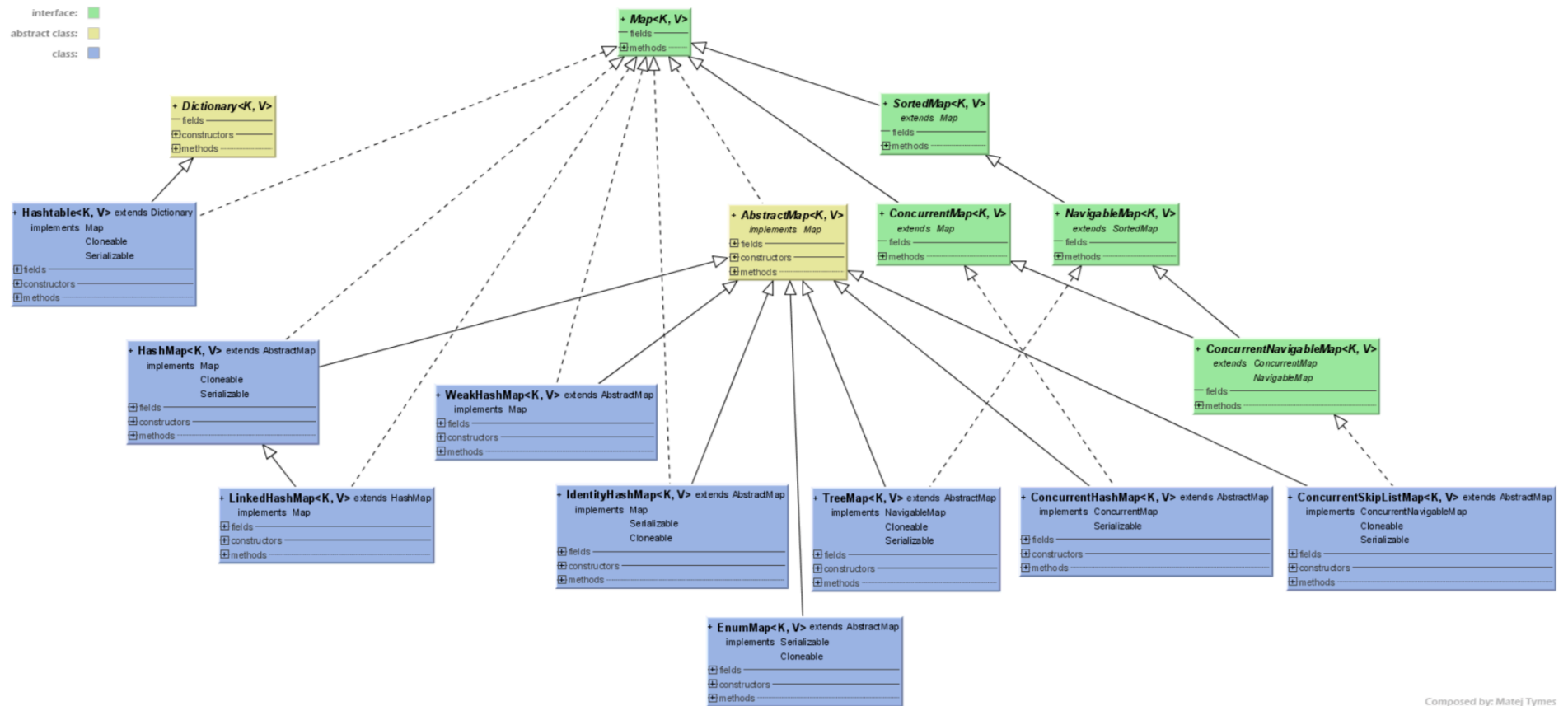
- Esta construção, ainda que muito poderosa, tem algumas limitações. A saber:
 1. como fazer que classes não relacionadas hierarquicamente possam ser vistas como tendo um tipo comum?
 2. como fazer para esconder, em determinados contextos, comportamento associado a um determinado tipo de dados?

- A solução passa por passar a utilizar um outro mecanismo de tipo de dados existente em Java: as **interfaces**.
- Interfaces não são classes, mas são apenas declarações sintáticas de expressão de comportamento
- não estão na mesma hierarquia das classes e possuem uma hierarquia própria de herança múltipla
- as classes decidem com que tipo de interfaces querem ser compatíveis

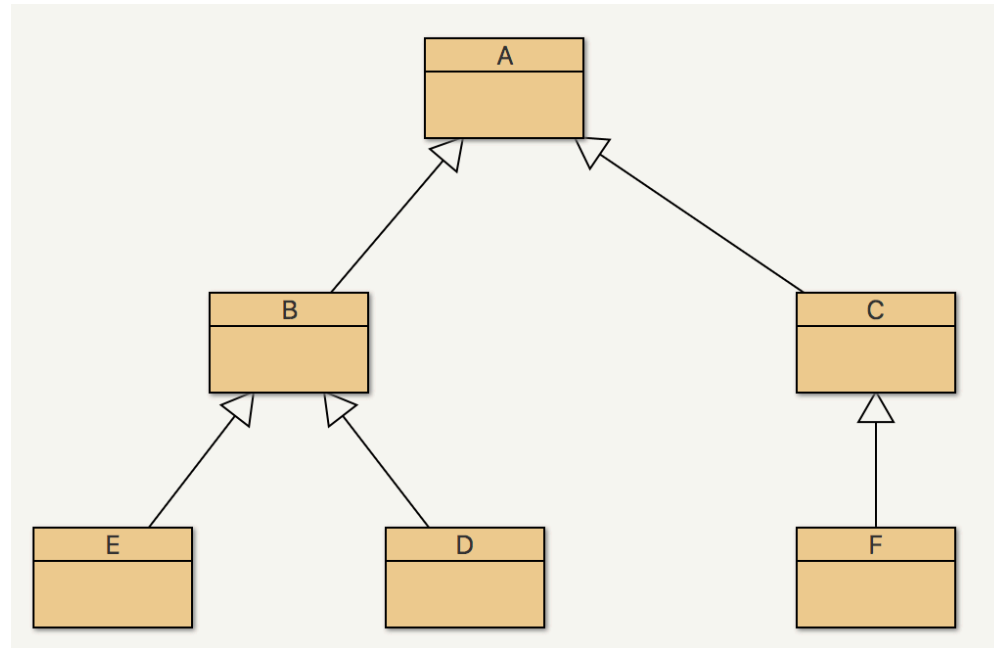
- Anteriormente já tivemos contacto com interfaces:
- **Set<T>, List<T>, Map<K, V>** não são classes, mas tipos de dados declarados como interfaces
- **Comparator<T>** é uma interface que representa o tipo de dados de todos os objectos que possuem (apenas!) o método `compare (...)` declarado
 - é uma functional interface



Composed by: Matej Tymes



- Consideremos as seguinte classes:

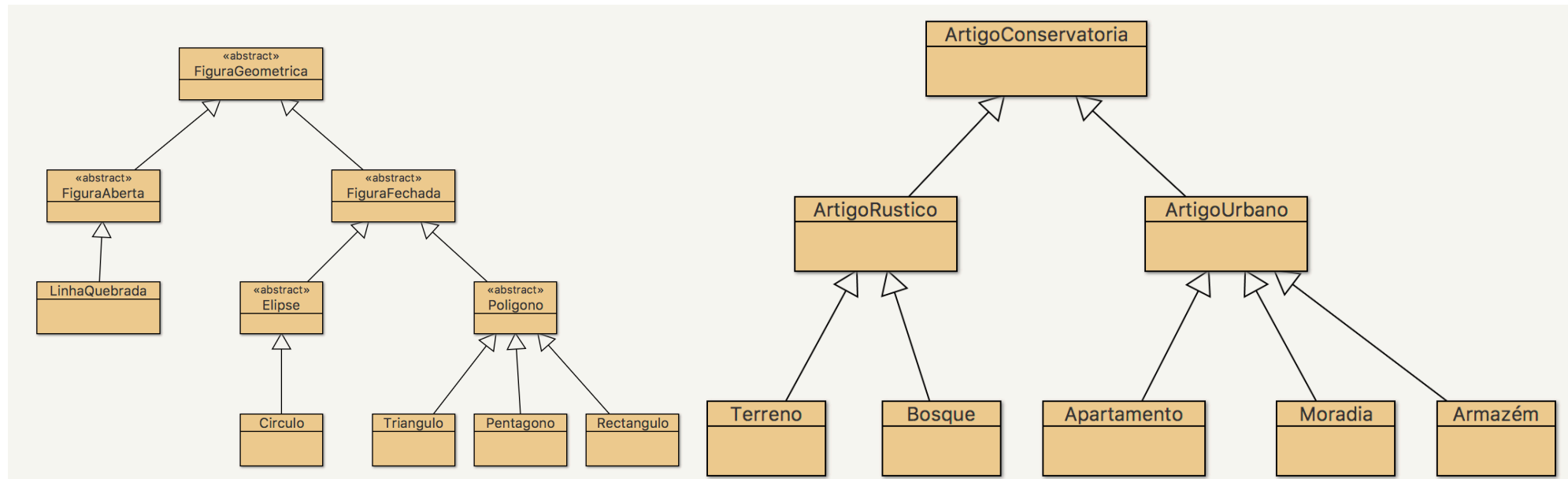


- Se quisermos representar uma colecção de objectos do tipo A (e dos seus subtipos) apenas precisamos de declarar:
- `Collection<A>`

- Mas o que acontece se quisermos apenas ter numa colecção objectos do tipo D e F?
- solução?: aceitamos objectos do tipo A e validamos depois se são D ou F? 💀
- solução: criamos um tipo de dados comum só a D e F!
- mas como o fazemos na construção hierárquica anterior?
- altera-se a hierarquia?

- Reescrever a hierarquia quase nunca é uma hipótese viável porque isso tem (mesmo muito!) impacto nas classes que usam objectos desta hierarquia
- é impossível fazer isso de forma silenciosa!
- pretendemos ter programação genérica e extensível, mas também estável e sem impactos nas aplicações já existentes.

- Outro exemplo:



- todas estas classes possuem o método `area()` e `perimetro()`, mas como é que as compatibilizamos por forma a serem tratadas de forma comum?

- Criando uma interface, por exemplo chamada `Aravel`, que contenha a declaração do comportamento desejado (tipo de dados)

```
public interface Aravel {  
    public double area();  
    public double perimetro();  
}
```

- tornando agora possível ter classes que queiram ser compatíveis com este novo tipo de dados.

- As classes que queiram ser compatíveis com a interface devem fazer:

```
public class Terreno extends ArtigoRustico implements Aravel {
```

- Uma classe pode implementar mais do que uma interface.

```
public class Terreno extends ArtigoRustico implements Aravel, Mensuravel {
```

```
public interface Mensuravel {  
    public double valorMercado();  
}
```

- Uma instância de `Terreno` pode ser vista como sendo:
 - do tipo de dados `Terreno` e tem acesso a todos os métodos
 - do tipo de dados `Aravel` e tem acesso *apenas* a `area()` e `perimetro()`
 - do tipo de dados `Mensuravel` e tem acesso *apenas* a `valorMercado()`

Elementos de uma interface

- A declaração de interface pode conter:
 - um conjunto de métodos abstractos
 - um conjunto de constantes
 - métodos concretos, designados por *default methods*
 - métodos de classe concretos, *static methods*

Hierarquia das Interfaces

- As interfaces podem estar colocadas numa estrutura hierárquica, que ao invés da das classes é uma hierarquia múltipla
- uma interface pode herdar de mais do que uma interface
- nessas situações é preciso ter atenção à herança de métodos com a mesma assinatura
 - `NomeInterface.nomeMetodo()`

Default Methods

- Até à versão 8 do Java as interfaces apenas podiam declarar assinaturas de métodos, que as classes implementadoras deviam tornar concretos.
- A partir dessa distribuição foi possível acrescentar métodos completamente codificados e que podem ser utilizados pelas classes que implementem a interface.

- Consideremos a interface `Aravel` definida anteriormente. O que acontece que acrescentarmos mais um método (abstracto) à interface?
- as classes que a implementam passam a dar erro de compilação, porque...
- ...falta implementar um método!

- Com a utilização dos *default methods* as classes que já implementam a interface não precisam de ser alteradas.
- As classes implementadoras, que o desejem!, passam a poder utilizar esse método
- Se a interface `Aravel` passar agora a ter um método default:

```
public interface Aravel {  
    public double area();  
    public double perimetro();  
    public default Ponto pontoCentral(...) { <<codigo>> }  
}
```

- A classe `Terreno` que implementa `Aravel` tem de implementar os métodos `area()` e `perimetro()` e pode utilizar o método `pontoCentral(...)` já codificado.
- As classes que não conhecem a implementação de `pontoCentral(...)` continuam inalteradas e a funcionar como até agora.

- Os default methods permitem adicionar funcionalidade às interfaces já existentes, sem alterar as relações de implementação que estão definidas
- os default methods foram criados para permitirem acrescentar funcionalidade à API das Collections
- Se a classe tiver um método com a mesma assinatura que um default method, prevalece a definição da classe!

Métodos static em interfaces

- É também possível criar métodos static na definição das interfaces.
- as instâncias das classes que implementam a interface tem acesso a estes métodos
- invocáveis da forma
`Interface.metodo()`

Classes abstractas e interfaces

- A escolha de utilização de uma classe abstracta ou uma interface é por vezes uma decisão difícil.
- Uma classe abstracta pode ter variáveis de instância enquanto uma interface não pode.
- Uma classe abstracta e uma interface podem declarar métodos abstractos para outras classes implementarem.

- No entanto, uma classe abstracta não obriga as subclasses a implementá-los. Se não o fizerem as subclasses ficam como abstractas. Nas interfaces, se não o fizerem, dá erro de compilação.
- Uma interface é um tipo de dados (uma API) que pode ser utilizada por qualquer classe. A definição da classe abstracta só é possível dentro da hierarquia.
- Em Java tem sido dada uma importância acrescida às interfaces como mecanismo de extensibilidade.

... e ainda

- A verificação de tipo pode ser feita da mesma forma que fazemos para as classes, com instanceof

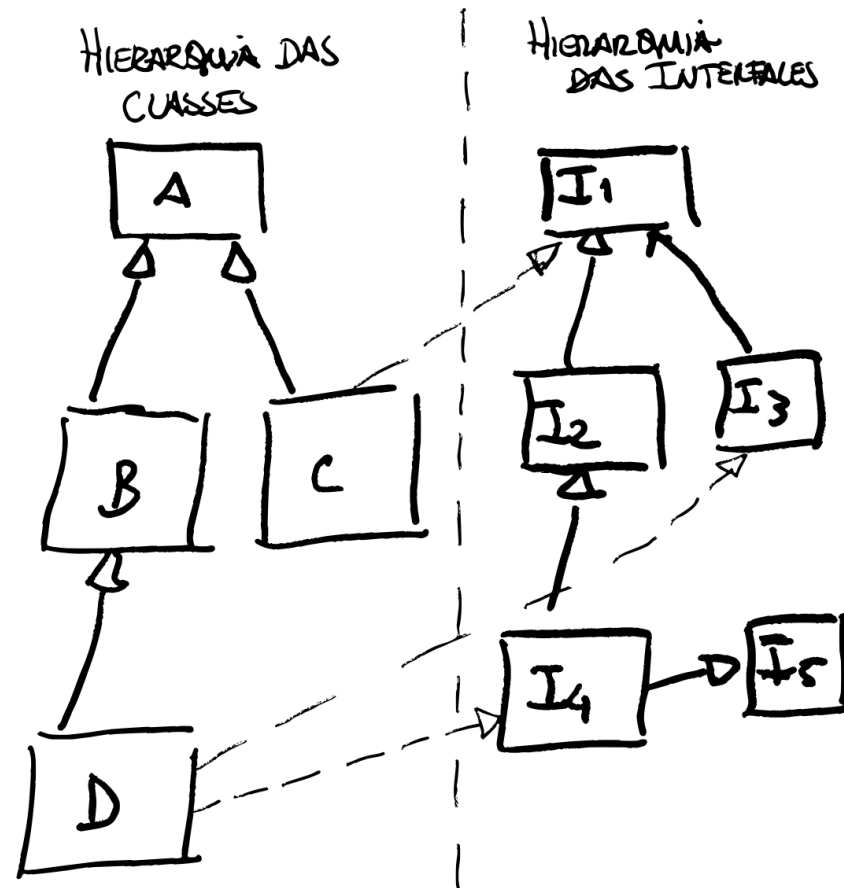
```
List<ArtigoConservatoria> propriedades = new ArrayList<>();  
...  
...  
int numMensuravel = 0;  
for(ArtigoConservatoria ac: this.propriedades)  
    if (ac instanceof Mensuravel)  
        numMensuravel++;
```

- na expressão acima não se está a validar a classe, mas sim o tipo de dados estático

- Do ponto de vista da concepção de arquiteturas de objectos, as interfaces são importantes para:
 - reunirem similaridades comportamentais, entre classes não relacionadas hierarquicamente
 - definirem novos tipos de dados
 - conterem a API comum a vários objectos, sem indicarem a classe dos mesmos (como no caso do JCF)

- Uma classe passa a ter duas vistas (ou classificações) possíveis:
- é subclasse, por se enquadrar na hierarquia normal de classes, tendo um mecanismo de herança simples de estado e comportamento
- é subtipo, por se enquadrar numa hierarquia múltipla de definições de comportamento (abstracto ou já implementado cf. default methods)

- existem duas hierarquias, que estão relacionadas, e o programador pode tirar partido disso.

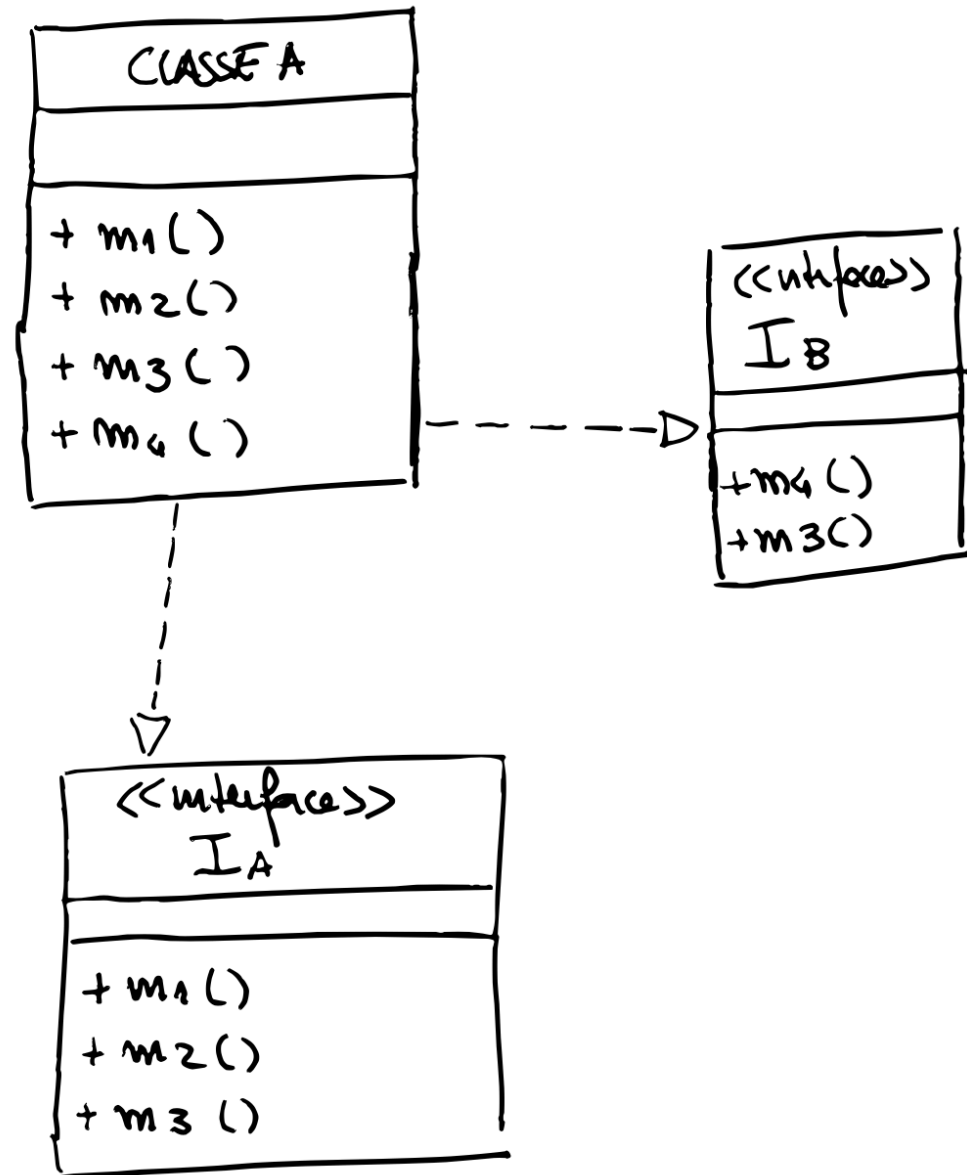


```
I1 obj = new C();
```

```
I4 outro = new D();
```

```
I3 aindaOutro = new D();
```

- as interfaces podem ser utilizadas para agrupar comportamento e limitar o acesso ao conjunto de métodos de uma classe
- IA apenas disponibiliza os métodos m1, m2 e m3



finalmente...

- Classes podem implementar múltiplas interfaces
- As interfaces podem:
 - incluir métodos **static**
 - fornecer implementações por omissão dos métodos (os métodos **default**)
- Functional Interface
 - uma interface com um único método abstracto (e qualquer número de métodos default)
 - Instâncias criadas com expressões lambda e com referências a métodos ou construtores

Em resumo...

- As interfaces Java são especificações de tipos de dados. Especificam o conjunto de operações a que respondem objectos desse tipo
- Uma instância de uma classe é imediatamente compatível com:
 - o tipo da classe
 - o tipo da interface (se estiver definido)

Interfaces

- permitem relacionar, sob a figura de um novo tipo de dados, objectos de classes não relacionadas hierarquicamente
- possibilita manter uma hierarquia de herança e ter uma outra hierarquia de tipos de dados

- possibilitam que um mesmo objecto possa apresentar-se sob a forma de diferentes tipos de dados, exibindo comportamentos diferentes, consoante o que seja pretendido
- permite esconder a natureza do objecto e fazer a sua *tipagem* de acordo com as necessidades do momento
- permite limitar os métodos que, em determinado momento, podem ser invocados num objecto

- (muito relevante) possibilitam que o código possa ser desenvolvido apenas com o recurso à interface e sem saber qual a implementação
- principal razão para termos utilizado `List<E>`, `Set<E>`, `Map<K,V>`
- o programador apenas necessita saber o comportamento oferecido e pode construir os seus programas em função disso

- As declarações constantes de uma interface constituem um contrato, isto é, especificam a forma do comportamento que as implementações oferecem
- Por forma a fazer programas apenas precisamos de saber isso e ter bem documentado o que cada método faz.
- Em função disso podemos fazer o programa e os programas de teste.